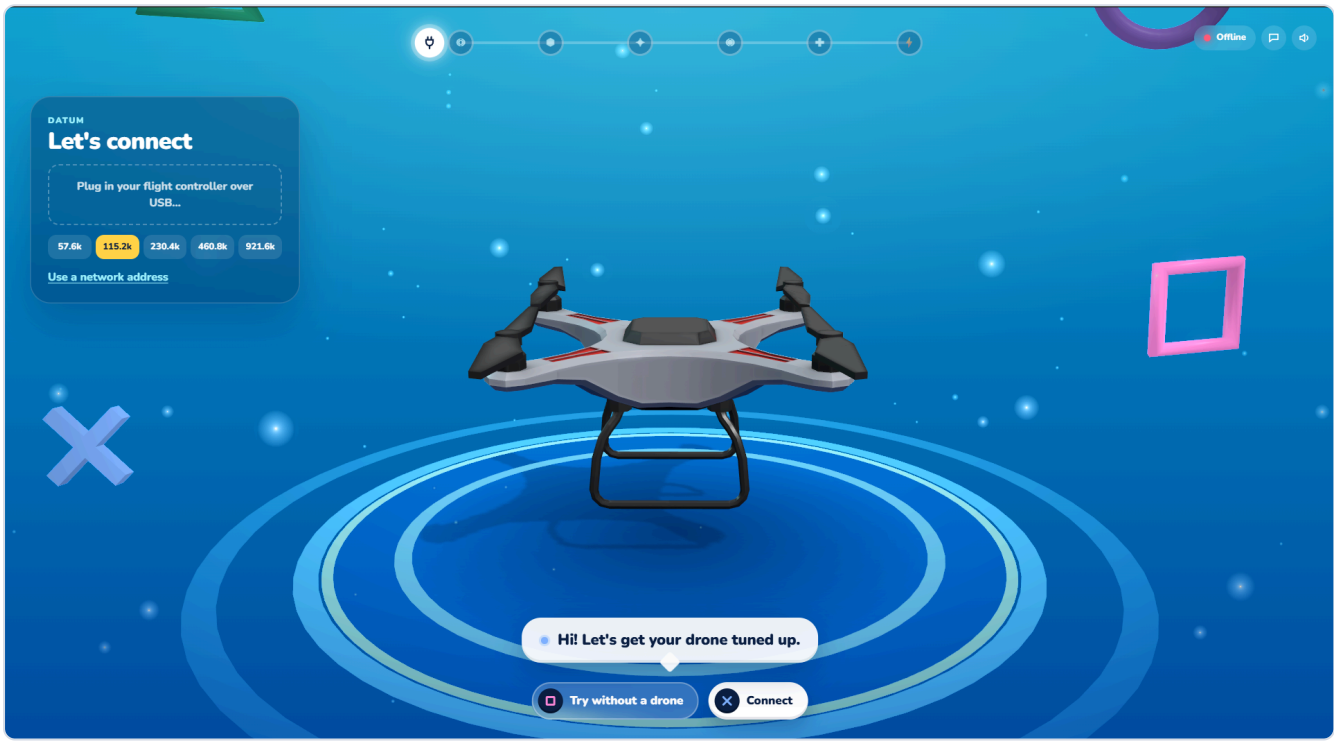
PROJECT REPORT

Datum

An interactive, game-style calibration tool for ArduPilot and PX4 drone flight controllers



Members

Name	Register No.
P Preetham	23BRS1346
Joel Anto	23BRS1165

Source: github.com/ppmpreetham/datum · October 2026

Contents

1. Abstract
2. Methodology
    1. Problem statement and objectives
    2. System architecture
    3. Technology stack
    4. Communication layer: MAVLink
    5. Calibration workflow design
    6. Coordinate frames and attitude mathematics
    7. Compass coverage algorithm
    8. Radio calibration algorithm
    9. ESC calibration and motor test
    10. Simulator and phone stand-in
    11. Safety and input validation
    12. User-experience design
    13. Testing methodology
    14. Codebase metrics
3. Outcome
    1. Summary of results
    2. Screenshot walkthrough
    3. Test results
    4. Limitations and future work
    5. Conclusion

1. Abstract

Before a multirotor can fly safely, its flight controller must be calibrated: the gyroscope bias, the accelerometer offsets and scale, the magnetometer (compass) offsets, the level reference, the radio-control channel ranges and the electronic speed controller (ESC) throttle endpoints. Ground-control stations such as QGroundControl and Mission Planner offer these procedures, but they present them as dense forms and text prompts, which beginners find hard to follow and easy to get wrong.

Datum is a cross-platform desktop application that turns calibration into a guided, playful 3D experience inspired by *Astro's Playroom*. A toy-like drone on a stage mirrors the real vehicle's attitude in real time, ghost poses and arrows show the user what to do next, and every step ends with a clear success screen. The application is built with Tauri 2 (a Rust backend) and React 19 with react-three-fiber (a TypeScript/WebGL frontend). It communicates with the flight controller over MAVLink v2 via USB serial, UDP or TCP, so no other ground-control software is needed. A built-in simulator reproduces every flow without hardware, so the tool can be used for training and demonstrations.

The outcome is a working application covering seven guided steps (connect, gyroscope, accelerometer, compass, level horizon, radio and ESC/motor test), 49 backend commands and 32 passing automated tests.

2. Methodology

2.1 Problem statement and objectives

Calibration errors are a leading cause of failed first flights: a compass calibrated near metal, an accelerometer pose held in the wrong orientation, or an ESC that never learnt its throttle range. Existing tools assume the user already understands the underlying procedure. The project set out to build a tool that *teaches while it calibrates*.

#	Objective	How it is measured
O1	Direct flight-controller link	Connects over MAVLink (serial/UDP/TCP) with no QGroundControl dependency
O2	Full calibration coverage	Gyro, accelerometer, compass, level, radio, ESC and motor test all implemented
O3	Visual, guided UX	Live 3D vehicle, pose hints and success or failure feedback on every step
O4	Firmware fidelity	Same order and semantics as QGroundControl (e.g. the six-pose order requested by the autopilot)
O5	Hardware-free mode	A simulator that implements the same backend interface
O6	Safety	Bounded motor tests, allow-listed parameters, session checks

2.2 System architecture

Datum follows a two-tier desktop architecture. The **frontend** owns presentation: the 3D scene, step logic, captions and sound. The **backend** owns the vehicle: the MAVLink connection, a dedicated worker thread, session state for each calibration, parameter reads and writes, and validation. The two halves communicate through Tauri's `invoke()` IPC and event channel. A small axum WebSocket server lets a phone stream its orientation into the app.

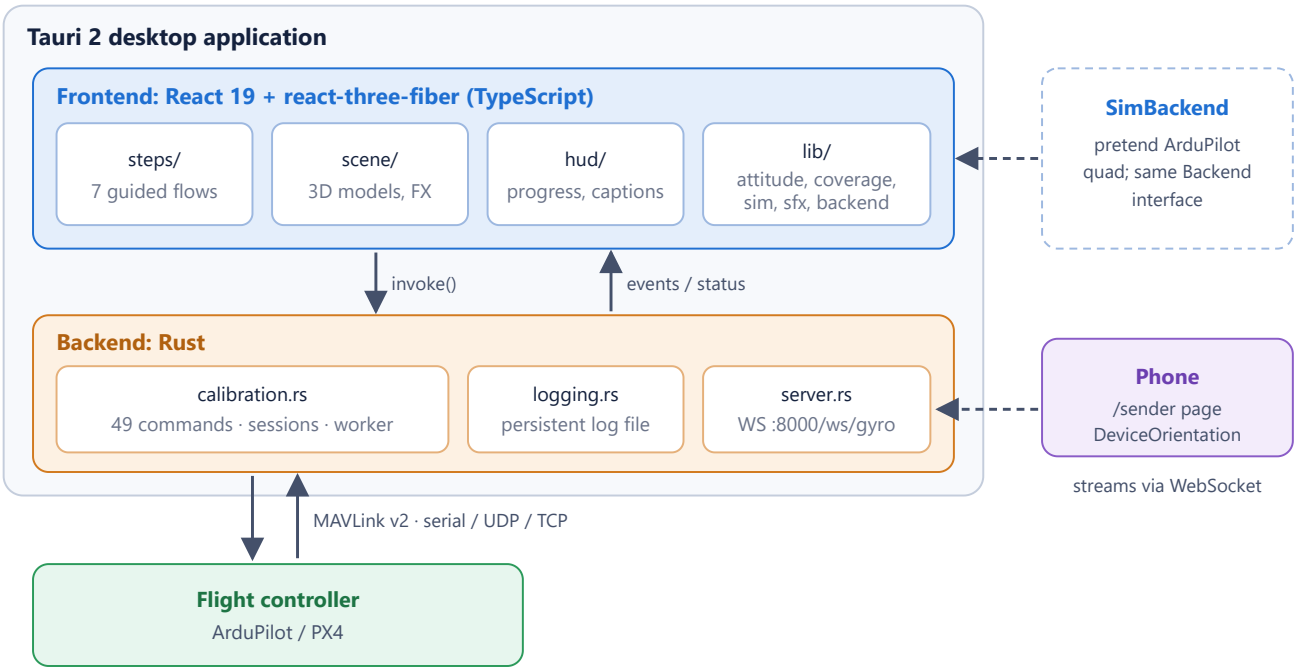


Figure 1. System architecture. Solid arrows: production data path. Dashed: hardware-free alternatives.

Both backends implement one TypeScript `Backend` interface (`src/lib/backend.ts`). The UI therefore never branches on whether hardware is present: `TauriBackend` forwards to Rust, and `SimBackend` (`src/lib/sim.ts`) emulates the same status streams in the browser.

Backend concurrency model

The Tauri command handlers never touch the serial port directly. They lock a shared `Session` (a `Mutex` behind `CalibrationService`), validate the request, and send a `WorkerCommand` to a dedicated vehicle thread (`run_vehicle`). That thread owns the MAVLink connection: it sends a 1 Hz ground-station heartbeat, issues commands, and parses every incoming message into session state (attitude, `SYS_STATUS` sensor health, `RC_CHANNELS`, calibration progress). Commands that must be confirmed, such as parameter writes, use a reply channel with a timeout. Each value is read back after it is written, so a dropped packet cannot be reported as success.

2.3 Technology stack

Layer	Technology	Purpose
Desktop shell	Tauri 2	Native window, IPC between Rust and the web view, small installer
Backend language	Rust 2021	Memory-safe, concurrent vehicle I/O
Vehicle protocol	`mavlink` 0.17 (ardupilotmega dialect)	MAVLink v2 encoding and decoding over serial, UDP and TCP
Serial discovery	`serialport` 4.10	Lists USB ports with vendor and product IDs
Async / WebSocket	tokio + axum 0.7	Local WebSocket server for the phone stand-in
UI framework	React 19 + TypeScript 5.8	Step components and state
3D rendering	three.js 0.185, react-three-fiber 9, drei, postprocessing	Drone, transmitter, ESC and compass-sphere scenes, bloom and particles
Animation	GSAP 3	Camera moves and UI transitions
Styling / build	Tailwind CSS 4, Vite 7	Styling, hot-reload development build
Audio	Web Audio API, SpeechSynthesis	Synthesized sound effects (no audio files) and spoken captions

Table 1. Technology stack.

2.4 Communication layer: MAVLink

MAVLink is the lightweight binary protocol used by ArduPilot and PX4. Datum identifies itself as a ground station (system 255, component 190) and drives calibration with the standard commands and messages below. Telemetry rates are requested explicitly with `MAV_CMD_SET_MESSAGE_INTERVAL`: `RC_CHANNELS` (id 65) at 10 Hz during radio setup, and `MAG_CAL_PROGRESS` / `MAG_CAL_REPORT` (ids 191 and 192) at 2 Hz during compass calibration.

Message / command	Direction	Use in Datum
`HEARTBEAT`	both	Detects the autopilot (ArduPilot or PX4), system ID and armed state; keeps the link alive
`ATTITUDE`	FC → app	Roll, pitch and yaw that drive the live 3D drone
`SYS_STATUS`	FC → app	Sensor present/healthy bitmasks and battery voltage
`RC_CHANNELS`	FC → app	Live PWM (µs) for up to 18 channels
`MAV_CMD_PREFLIGHT_CALIBRATION`	app → FC	Gyro (p1=1), accel (p5=1), level (p5=2), PX4 compass (p2=1), PX4 radio (p4=1)
`MAV_CMD_ACCELCAL_VEHICLE_POS`	app → FC	Confirms each of the six accelerometer poses (1 to 6)
`MAV_CMD_DO_START_MAG_CAL` / `ACCEPT` / `CANCEL`	app → FC	ArduPilot onboard compass calibration (compass mask, retry, autosave)
`MAG_CAL_PROGRESS`, `MAG_CAL_REPORT`	FC → app	Per-compass completion percentage and final fitness
`PARAM_REQUEST_READ`, `PARAM_SET`, `PARAM_VALUE`	both	Frame, orientation, `RCn_MIN/MAX/TRIM`, `MOT_PWM_TYPE`, failsafe and battery settings
`MAV_CMD_DO_MOTOR_TEST`	app → FC	Spins one motor at a bounded throttle for a bounded time
`COMMAND_ACK`, `STATUSTEXT`	FC → app	Command acceptance and autopilot prompts (e.g. "Place vehicle on its LEFT side")

Table 2. MAVLink messages and commands used.

Supported transports

Address form	Example	Constraint
Serial	`serial:COM3:115200`	Baud rate 57.6k to 921.6k (chosen on the connect screen)
UDP listen	`udpin:127.0.0.1:14550`	Loopback only, so the app never listens on the network
UDP / TCP client	`tcpout:127.0.0.1:5760`	For SITL simulators and telemetry bridges

Table 3. Connection address formats.

2.5 Calibration workflow design

The workflow is a fixed sequence of seven steps, shown as a progress track at the top of the screen. Each sensor step opens with the same "Before we start" card, which confirms the board orientation, mirroring QGroundControl. It then runs the calibration and ends in a success state (with "Do it again" / "Next") or a failure state that explains what went wrong.

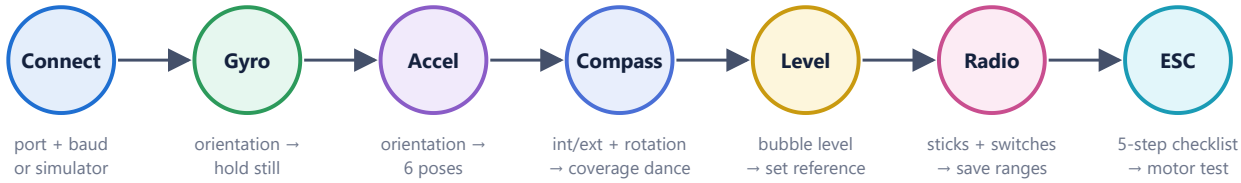


Figure 2. The seven-step guided flow.

Step	Firmware mechanism	User action / visual guidance	Success criterion
Gyroscope	`PREFLIGHT_CALIBRATION` p1 = 1	Keep the vehicle still; the frontend measures angular speed from attitude samples	`COMMAND_ACK` accepted
Accelerometer	`PREFLIGHT_CALIBRATION` p5 = 1, then `ACCELCAL_VEHICLE_POS` 1–6	Level, left, right, nose-down, nose-up, upside-down; a ghost drone and arrow show each target pose	Autopilot reports calibration successful after pose 6
Compass	ArduPilot `DO_START_MAG_CAL` (mask, retry, autosave); PX4 p2 = 1	Rotate through all orientations; a coverage sphere fills in and an arrow suggests the next turn	`MAG_CAL_REPORT` success; fitness shown on a green/yellow/red bar
Level horizon	`PREFLIGHT_CALIBRATION` p5 = 2	Place on a level surface; 3D bubble level with live roll/pitch read-outs	ACK accepted; new level reference stored
Radio	Capture `RC_CHANNELS`; write `RCn_MIN/MAX/TRIM`	3D FS-i6 transmitter mirrors each stick and switch	Each used channel travels ≥ 400 µs; parameters read back and verified
ESC	Read `MOT_PWM_TYPE`; guided checklist; `DO_MOTOR_TEST`	Battery, ESC and motor model; step checklist; motor A–D picker	All five steps confirmed in order; each motor spins in the expected position

Table 4. Calibration steps, mechanisms and success criteria.

Session state machine

Every multi-stage calibration (accelerometer, compass, radio and ESC) is guarded by a session ID generated by the backend (e.g. `radio-<generation>-<millis>`). Subsequent commands must present the same ID, so a stale UI or a second window cannot advance or cancel someone else's calibration. The backend exposes the phases *idle* → *running/capturing* → *succeeded* | *failed* | *cancelled* and the frontend renders each phase directly.

2.6 Coordinate frames and attitude mathematics

MAVLink reports attitude in the aerospace **NED** convention (x forward, y right, z down), as body-to-NED Euler angles applied in Z-Y-X (yaw, pitch, roll) order. The three.js scene uses a **Y-up** frame with the drone's nose pointing away from the camera (x right, y up, z towards the viewer). The mapping between the two is a fixed permutation:

$$(x, y, z)_{\mathrm{NED}} \mapsto (y, -z, -x)_{\mathrm{three}}$$

Writing that permutation as a rotation C, an attitude quaternion is carried across by conjugation, which keeps rotations composable:

$$q_{\mathrm{three}} = q_C \cdot q_{\mathrm{NED}} \cdot q_C^{-1}$$

To keep the drone facing the camera however the real vehicle is oriented, the yaw at first contact is stored as `yawOffset` and removed before display. The same transform is inverted when the user drags the model in simulator mode, so dragging and real telemetry write to one shared attitude state. Angular speed (used for the "hold still" check) is estimated from consecutive samples with exponential smoothing:

$$\hat{\omega} \leftarrow \hat{\omega} + (\theta/\Delta t - \hat{\omega}) \cdot \min(1, 6\Delta t), \quad \theta = \angle(q_{\mathrm{prev}}, q)$$

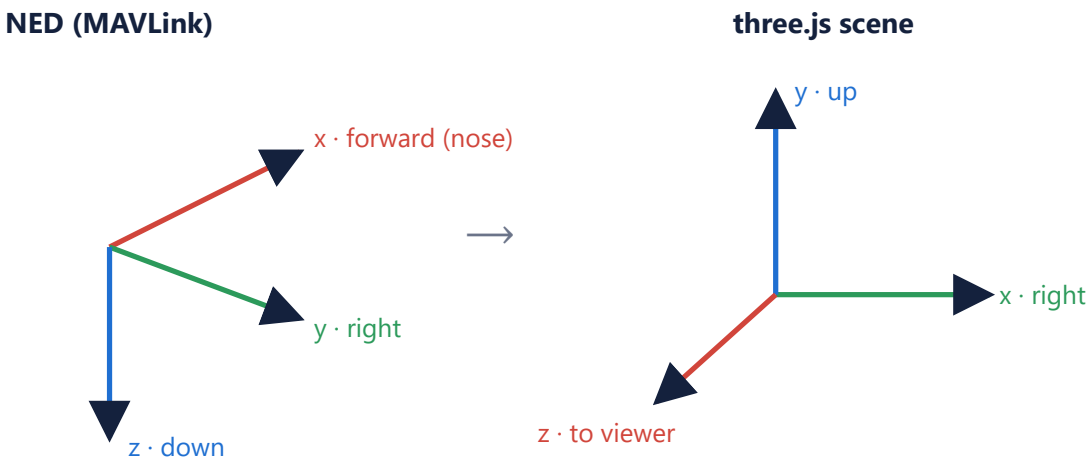


Figure 3. Reference frames. Colours mark corresponding physical axes: forward ↔ −z, right ↔ x, down ↔ −y.

2.7 Compass coverage algorithm

A magnetometer calibration fits a sphere (or ellipsoid) to field samples, so it needs Earth's magnetic field to have been seen from as many body-frame directions as possible. Datum makes this visible. It places **110 cells** evenly on a unit sphere with the Fibonacci (golden-angle) lattice:

$$y_i = 1 - 2i/(N-1), \quad r_i = \sqrt{(1 - y_i^2)}, \quad \theta_i = i \cdot \pi(3 - \sqrt{5})$$

Each frame, the Earth field vector (modelled with a 60° inclination) is rotated into the body frame. Every unpainted cell within **21°** of it is painted. The drone is drawn inside a translucent geodesic sphere whose cells light up as they are covered, so the user sees which orientations are still missing.

To guide the user, the **hint** step scores each unpainted cell by the number of unpainted neighbours within 45°, plus a bias (4 × cosine) toward cells near the current field direction. It takes the cross product of the target direction and the field to get a rotation axis, then names the dominant body axis (roll, pitch or yaw)

and its sign. The arrow and caption ("Roll it to the left!") come from that result. Cumulative rotation about each body axis is tracked from quaternion deltas and shown as three small Yaw/Pitch/Roll rings.

Parameter	Value	Rationale
Sphere cells	110	Fine enough to show gaps, cheap enough for $O(N^2)$ hint scoring every frame
Paint radius	21°	Neighbouring cells overlap, so a smooth sweep leaves no holes
Neighbourhood for hints	45°	Favours turning toward large unexplored patches
Field inclination	60°	Typical mid-latitude dip angle

Table 5. Coverage-sphere parameters.

The firmware's own progress (`MAG_CAL_PROGRESS`) remains the authority on completion. The coverage sphere is guidance, and the backend labels its text hints as `backend_heuristic`, banded by progress: 0–19 % level heading circle, 20–39 % nose up, 40–59 % nose down, 60–79 % each side, 80 % and above upside-down sweep.

2.8 Radio calibration algorithm

When capture starts, every channel's minimum, maximum and trim are reset to its current value. During capture each `RC_CHANNELS` sample widens the min/max envelope. On finish, each channel is classified:

Classification	Rule (travel = max − min)	Action
Valid range	travel ≥ 400 µs	Write `RCn_MIN`, `RCn_MAX`, `RCn_TRIM`
Untouched / unused	travel < 100 µs	Skipped and left unchanged (e.g. iBUS channels 7–14 on a six-channel transmitter)
Partial	100 µs ≤ travel < 400 µs	Rejected: the user is asked to move that channel through its full range

Table 6. Radio channel classification.

If the user selected specific channels, the selection must be non-empty, contain only channels 1–18, and every selected channel must have live telemetry; otherwise the backend returns `INVALID_REQUEST` or `NO_RC_SIGNAL`. The parameters are written in one batch and read back before success is reported.

2.9 ESC calibration and motor test

Datum first reads `MOT_PWM_TYPE` to decide whether throttle-endpoint calibration applies at all. Digital protocols (DShot 150–1200) need none; analogue protocols (normal PWM, OneShot, OneShot125) do.

MOT_PWM_TYPE	Protocol	Endpoint calibration
0	Normal PWM	Required
1 / 2	OneShot / OneShot125	Required
3	Brushed	Not applicable
4–7	DShot150 / 300 / 600 / 1200	Not required (digital)

Table 7. ESC protocol detection.

The calibration itself is a strictly ordered five-step checklist: **remove propellers → disconnect battery → raise throttle → power up and confirm tones → lower throttle**. The backend accepts only the step it currently expects, so steps cannot be skipped or repeated. The motor test then spins motors A–D one at a time, highlighted on a 3D quad. Throttle is limited to **1–15 %** and duration to **0.1–5 s**; anything outside those bounds is rejected before it reaches the vehicle.

2.10 Simulator and phone stand-in

`SimBackend` is a scripted ArduPilot quad written in TypeScript. It reads the same shared attitude state that real telemetry writes to, so its calibrations react to what the user does. Dragging the screen tilts the drone (Shift rolls); the accelerometer only accepts a pose when the drone really is in it, and compass progress grows with sphere coverage. The 3D FS-i6 transmitter's sticks can be dragged and its switches clicked, producing simulated PWM on eight iBUS channels.

For a physical feel without a drone, a phone opens the `/sender` page. It streams `DeviceOrientation` (α, β, γ) over a WebSocket to the backend's axum server at `:8000/ws/gyro`, which rebroadcasts it to the app. There it

is converted from the phone's ENU, Z-X′-Y″ convention into a body-to-NED quaternion. A phone lying screen-up with its top pointing forward maps to a level drone.

2.11 Safety and input validation

Because the tool can spin motors and rewrite flight-critical parameters, the backend validates every request regardless of what the UI allows.

Risk	Mitigation
Unintended motor spin	Motor test bounded to motors 1–32, 1–15 % throttle, 0.1–5 s; ESC flow begins with "remove propellers"
Corrupting parameters	Setup parameter API is allow-listed; read-only names are refused for writes; values must be finite, non-negative integers; every write is read back
Stale or concurrent UI	Session IDs on accelerometer, compass, radio and ESC workflows; out-of-order ESC steps rejected
Network exposure	WebSocket server binds to 127.0.0.1 by default; UDP listen accepted only on loopback
Wrong firmware path	Commands chosen per autopilot (e.g. ArduPilot compass uses `DO_START_MAG_CAL`; ArduPilot radio is not sent through `PREFLIGHT_CALIBRATION`)
Silent failures	Typed error codes (`NO_RC_SIGNAL`, `INVALID_RC_RANGE`, `INVALID_SESSION`…) shown as captions; persistent backend log file

Table 8. Safety measures.

2.12 User-experience design

- **Controller grammar.** Every action maps to a PlayStation-style glyph: ✕ confirm (Enter/Space), ○ back or cancel (Esc), □ secondary (S). Floating ✕ ○ △ □ shapes decorate the stage.
- **Colour-coded progress.** Each step owns a colour (gyro green, accelerometer purple, compass blue, level gold, radio pink, ESC cyan), shown on the progress track and in that step's effects.
- **Show, don't tell.** Ghost poses, curved arrows, a bubble level, a coverage sphere and a living transmitter replace most of the instruction text.
- **Short, friendly captions.** One sentence at a time ("Roll it onto its left side."), optionally spoken through the browser's speech synthesis.
- **Feedback.** Synthesized tones for each event (no audio files), plus a confetti burst and a large tick card on success.
- **Camera choreography.** GSAP animates the camera to frame the subject of each step: the flight controller, the GPS mast, the transmitter, or the ESC bench.

2.13 Testing methodology

Backend logic is unit-tested with Rust's built-in test harness (`cargo test`). The tests target pure functions extracted from the command handlers, so they run without hardware: command parameter encoding, validation domains, range classification and workflow ordering. Frontend flows were verified by running the full sequence end-to-end in simulator mode (Section 3.2).

Area	Representative tests
Command encoding	`gyro_and_px4_radio_commands_are_distinct`, `motor_test_uses_percent_throttle_and_bounded_duration`
Accelerometer	`accelerometer_position_requests_have_distinct_prompts`
Compass	`ardupilot_compass_calibration_autosaves_after_success`, `compass_guidance_is_dynamic_and_explicitly_heuristic`
Radio	`radio_ranges_require_meaningful_travel`, `untouched_radio_channels_are_skipped_not_rejected`, `radio_selection_ignores_unselected_fixed_channels`
ESC / safety	`completed_or_cancelled_esc_workflow_cannot_advance`, `motor_operations_are_disabled`, `setup_parameter_api_is_allowlisted`
Connectivity	`connection_rejects_remote_bind_and_invalid_addresses`, `websocket_is_local_only_by_default`, `unsupported_autopilot_is_rejected`
Contracts	`manifest_matches_registered_tauri_commands_without_duplicates`, `serial_port_json_uses_frontend_field_names`

Table 9. Unit-test coverage by area.

2.14 Codebase metrics

The project totals roughly **12,500 lines** of source: about 6,800 lines of Rust and 5,700 of TypeScript/TSX. Figure 4 breaks this down by module. The backend's `calibration.rs` is the largest unit because it holds all 49 commands, the vehicle worker and its tests. On the frontend, most of the code is in the step flows and the 3D scene.

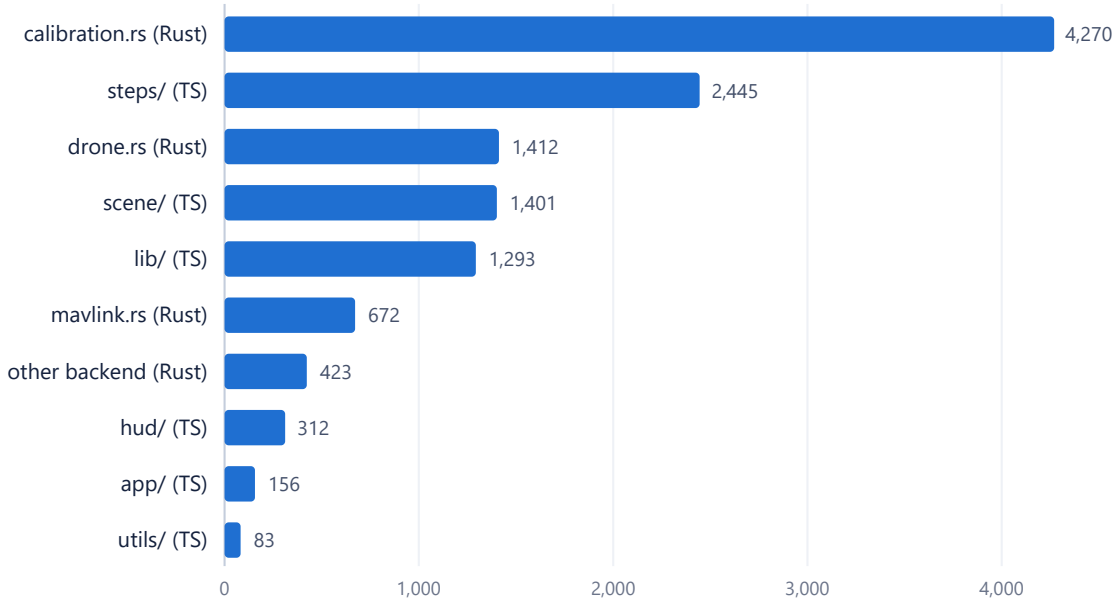


Figure 4. Source lines by module (Rust backend and TypeScript frontend).

Module	Language	Lines	Responsibility
calibration.rs	Rust	4,270	Commands, sessions, vehicle worker, validation, unit tests
drone.rs	Rust	1,412	Standalone MAVLink monitor / vehicle helpers
mavlink.rs	Rust	672	Typed heartbeat, status and telemetry decoding
server, logging, manifest, lib, backend, main	Rust	423	WebSocket, persistent log, endpoint manifest, app wiring
steps/	TSX	2,445	Seven guided step components and orientation setup
scene/	TSX	1,401	Stage, effects, drone, flight controller, ESC and radio models
lib/	TS	1,293	Attitude maths, coverage sphere, simulator, sound, backend bridge
hud/, app/, utils/	TSX	551	Progress track, captions, controls, global state, phone gyro

Table 10. Module responsibilities and size.

3. Outcome

3.1 Summary of results

All seven planned steps were implemented and run end-to-end. Every objective in Section 2.1 was met in simulator mode, and the backend's command layer is covered by automated tests. The gyroscope flow is the most mature; the other flows work in full against the simulator and still need validation on real drone hardware.

Component	Status	Result
Connect	Complete	Serial port and baud picker, network address entry, "Try without a drone" simulator
Gyroscope	Complete	Orientation check, stillness detection, calibration and success screen
Accelerometer	Complete · simulator-tested	Six-pose sequence in the autopilot's order with ghost-pose guidance
Compass	Complete · simulator-tested	Internal/external selection, coverage sphere, hints, fitness bar, reboot reminder
Level horizon	Complete · simulator-tested	Live roll/pitch read-out and 3D bubble level
Radio	Complete · simulator-tested	Live FS-i6 model, range capture, unused-channel handling, verified parameter save
ESC / motor test	Complete · simulator-tested	Protocol detection, ordered five-step checklist, bounded per-motor spin test
Automated tests	32 / 32 passing	22 library tests + 10 MAVLink-monitor tests

Table 11. Outcome summary.

3.2 Screenshot walkthrough

The following screenshots were captured from a complete run of the application, from connection to motor test.

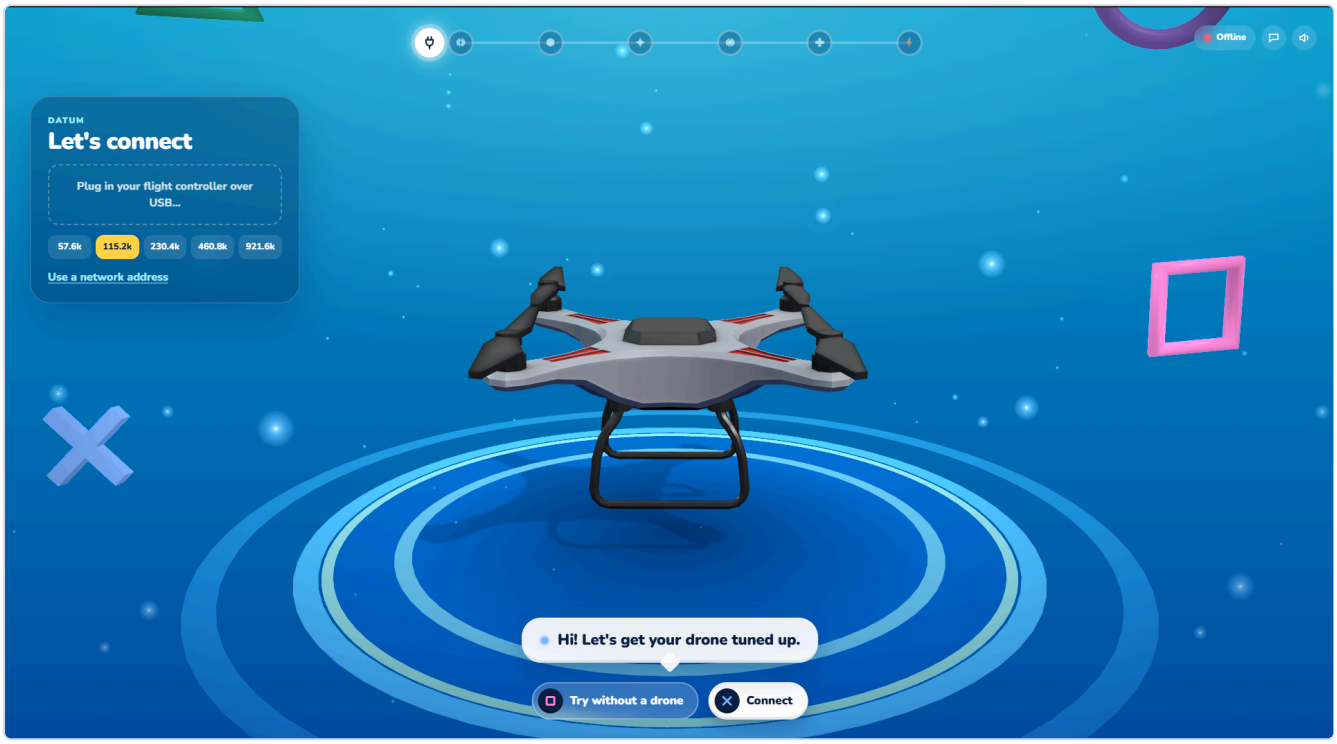


Figure 5. **Connect.** The welcome screen with the seven-step progress track (top), connection status (top right), USB baud-rate selection and network-address option. "Try without a drone" (□) starts the simulator; "Connect" (✕) opens the selected port.



Figure 6. **Autopilot rotation.** Before the gyroscope step, the camera zooms to a translucent drone with the flight controller highlighted. The gold arrow shows the board's forward direction and the drop-down sets `AHRS_ORIENTATION`.



Figure 7. **Gyroscope result.** "Gyro is wide awake!": the step turns green on the progress track, confetti plays, and the user can repeat (○) or continue (✕).



Figure 8. **Accelerometer.** The six-pose strip (Flat ✓, Left side active) follows the autopilot's requested order. A purple ghost drone shows the target pose and the arrow shows the direction to roll it.

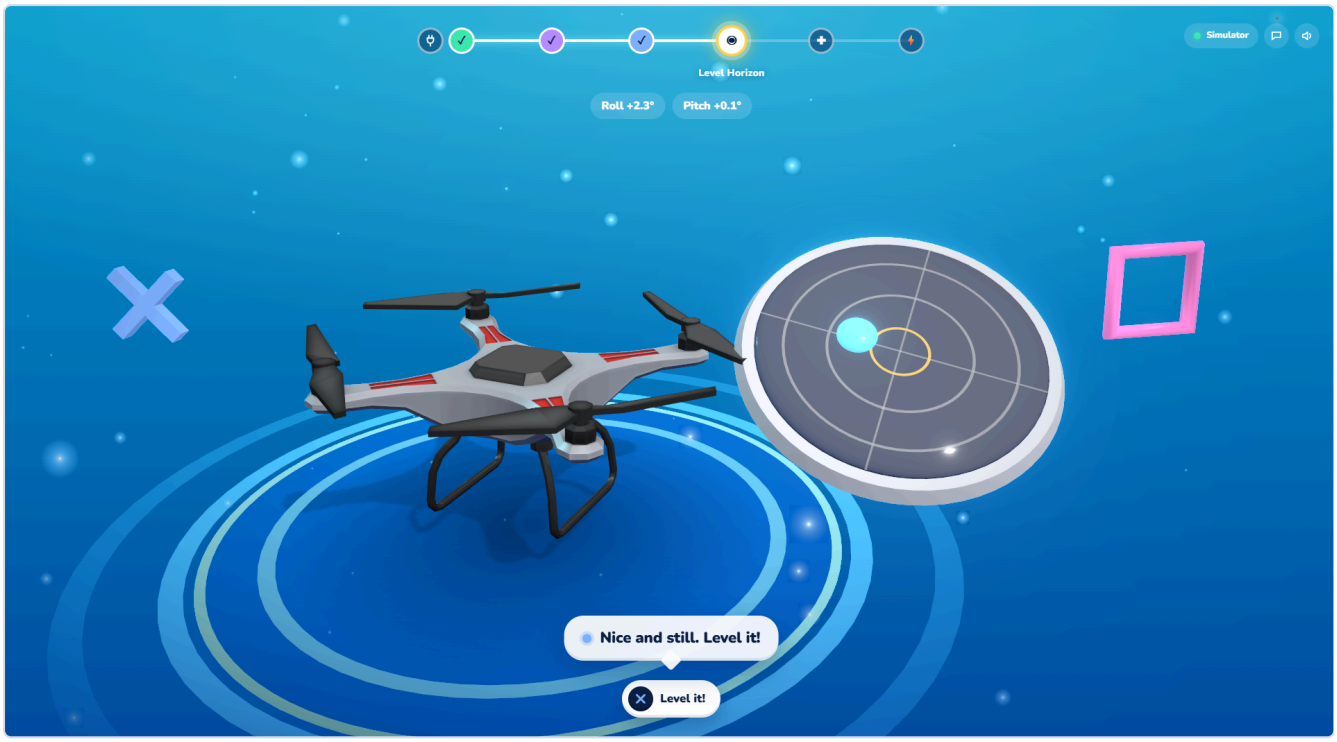


Figure 9. **Level horizon.** Live roll (+2.3°) and pitch (+0.1°) read-outs and a 3D bubble level. The user presses "Level it!" once the bubble settles inside the gold ring.



Figure 10. **Compass setup.** Choose an internal (flight-controller) or external (GPS puck on a mast) compass. For external units, the puck's arrow direction sets `COMPASS_ORIENT`.

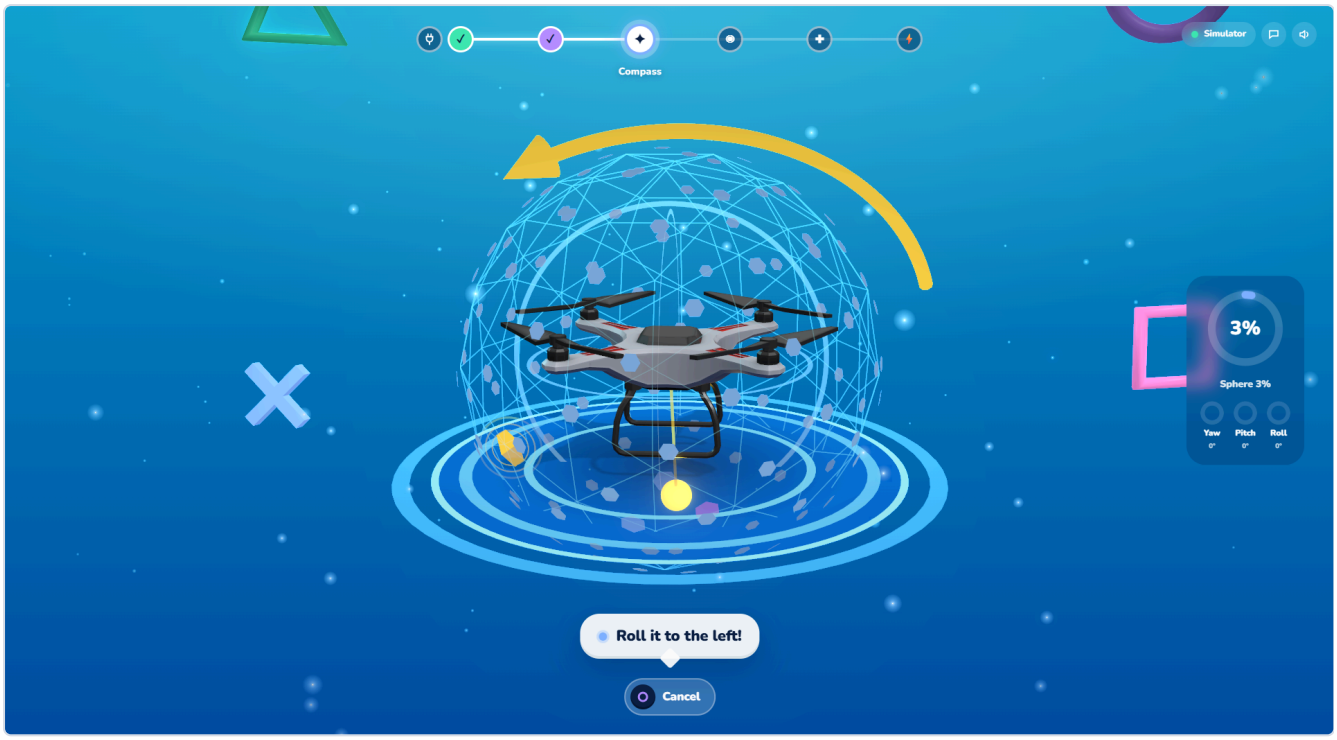


Figure 11. **Compass coverage dance.** The drone sits inside the 110-cell coverage sphere, and cells light up as the field direction (yellow ray and brush) sweeps over them. The gold arrow and caption ("Roll it to the left!") come from the hint algorithm; the panel on the right shows overall sphere coverage and per-axis Yaw/Pitch/Roll rings.

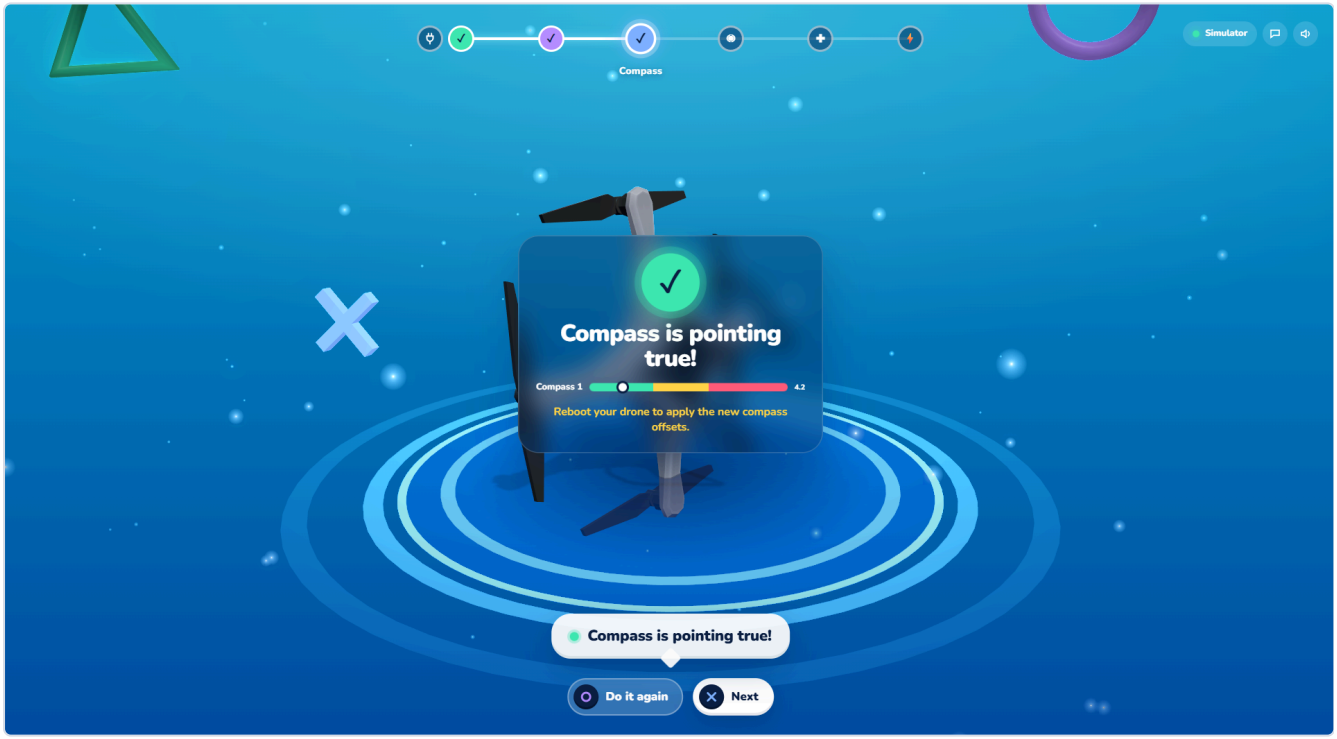


Figure 12. **Compass result.** "Compass is pointing true!" with QGroundControl-style fitness bar for Compass 1 (green < 8, yellow < 15, red beyond) and a reminder to reboot so the new offsets take effect.



Figure 13. **Radio.** A 3D FlySky FS-i6 transmitter and FS-iA6B receiver. The sticks, switches and on-screen channel bars mirror live `RC_CHANNELS` data; the side card confirms that a signal has been received.

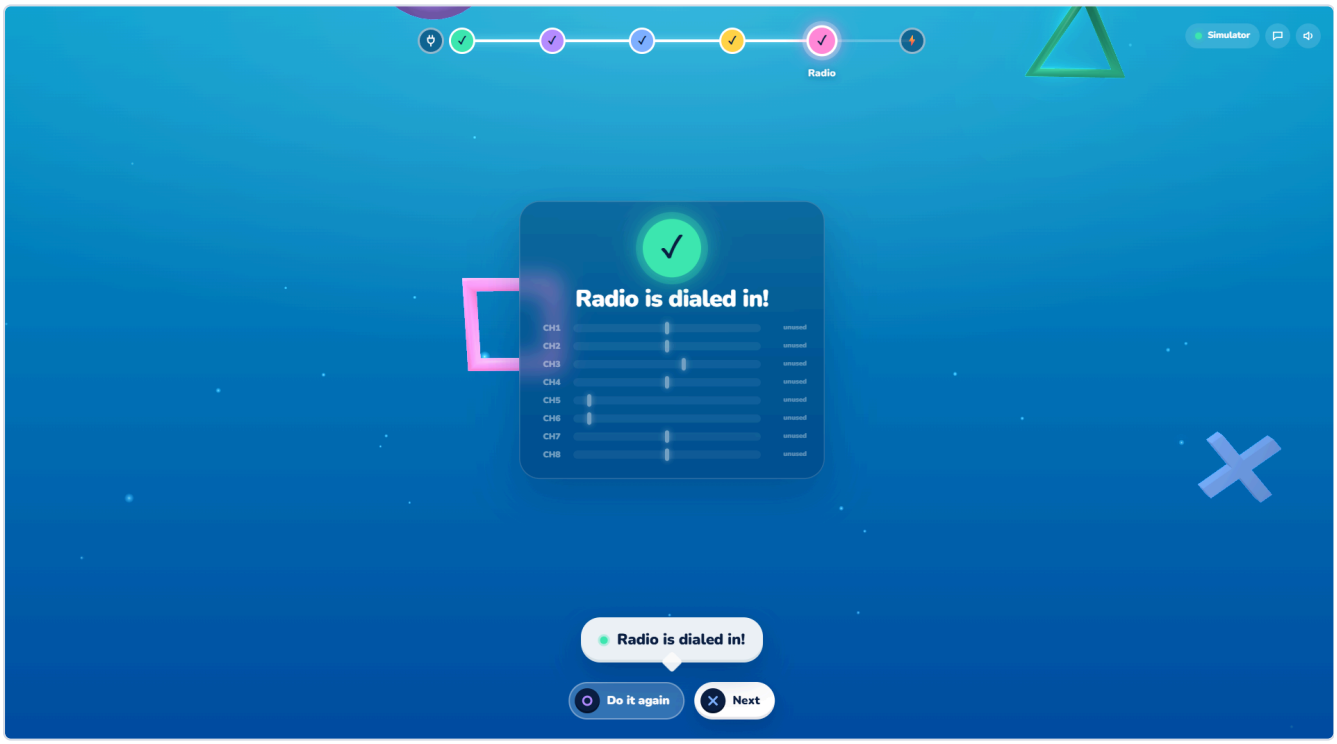


Figure 14. **Radio result.** "Radio is dialed in!" with the captured position of channels 1–8. Channels that did not move are marked *unused* and left untouched, so they don't block the save.



Figure 15. **ESC.** The detected protocol (Normal PWM, from `MOT_PWM_TYPE`) decides whether endpoint calibration is needed. The bench shows battery → XT60 connector → ESC → motor. The user can calibrate the ESCs (✕) or go straight to the motor test (□).



Figure 16. **ESC checklist, step 2.** The ordered checklist (Props off ✓ → Unplug battery → Throttle up → Power up → Throttle down) is on the left; the battery glows to show what to unplug.

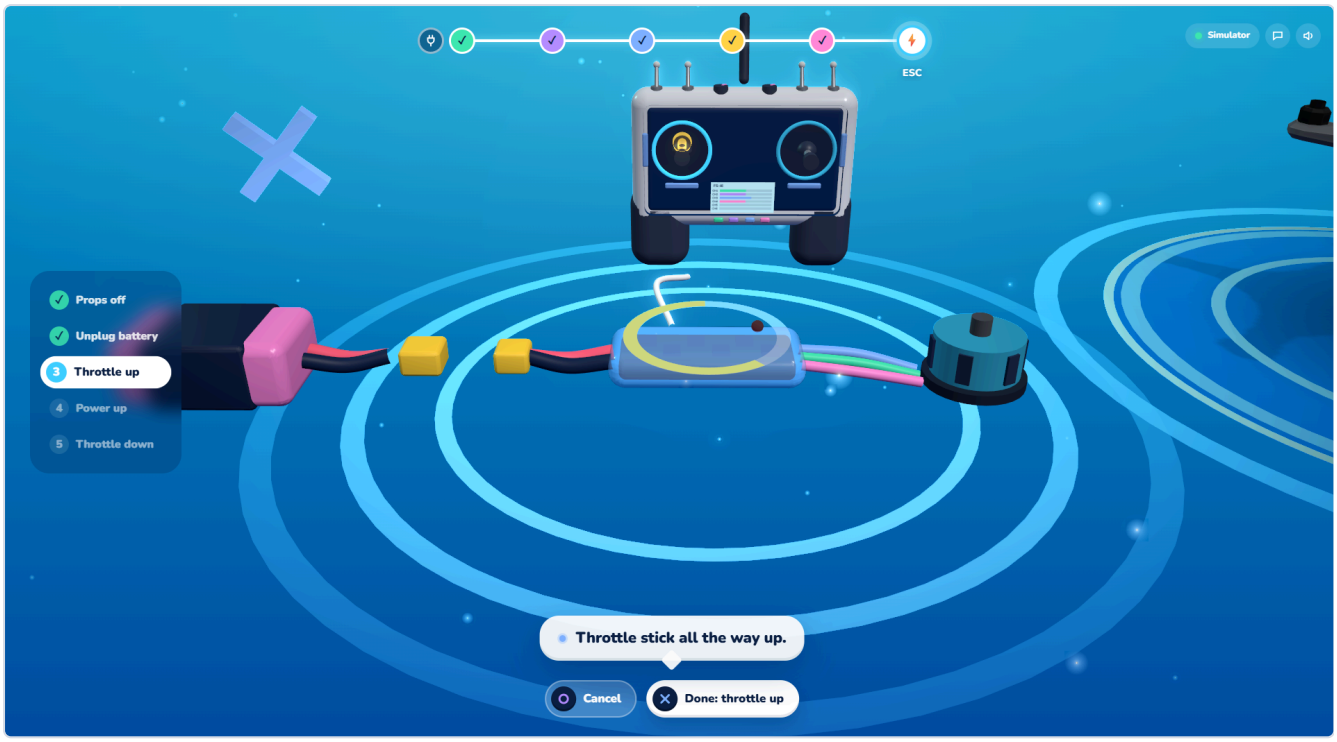


Figure 17. **ESC checklist, step 3.** The XT60 connector is shown separated and the transmitter appears with its throttle stick highlighted, prompting "Throttle stick all the way up."



Figure 18. **Motor test.** After calibration ("ESCs know their range now!") the user picks motor A–D on a quad-X diagram, which shows each motor's spin direction. Throttle (8 %) and time (2 s) sliders stay within the backend limits, and the selected motor is ringed on the 3D drone.

3.3 Test results

The backend test suite was run with `cargo test` after the final merge. All tests pass.

Test binary	Tests	Passed	Failed
datum_lib (calibration, logging, server, endpoint manifest)	22	22	0
mavlink_monitor (MAVLink decoding and vehicle helpers)	10	10	0
Total	**32**	**32**	**0**

Table 12. Automated test results.

3.4 Limitations and future work

- **Hardware validation.** The flows have been exercised end-to-end in the simulator. The main next step is validating each one on real drone hardware, starting with the accelerometer, compass, level and ESC flows, which the project tracker marks as in progress.
- **QGroundControl integration.** QGroundControl's MAVLink forwarding is outbound only, so Datum cannot share its link. Datum currently replaces QGroundControl during setup rather than running alongside it, and only one application can own the USB port.
- **Compass fitness.** The fitness value is shown only when the firmware's `MAG_CAL_REPORT` provides it; the coverage sphere is a guide, not a substitute for the onboard fit.
- **Transmitter models.** The 3D radio is modelled on the FlySky FS-i6; other transmitters work but are drawn as the FS-i6.

3.5 Conclusion

Datum shows that drone calibration does not have to be intimidating. By talking MAVLink directly to the flight controller and presenting each procedure as a short, visual, game-like sequence, it gives beginners the same calibrations a ground-control station provides with far more guidance. Its layered architecture, with a validated and session-guarded Rust backend, a frontend that works with either the real vehicle or the simulator, and a careful mapping between the two coordinate frames, keeps the playful presentation separate from the safety-critical vehicle logic.